

Projektstruktur & EEG-Logik

Dieses Dokument bietet einen detaillierten Überblick über die Architektur des Projekts, insbesondere über das Zusammenspiel der Brain-Computer Interface (BCI) Komponenten mit dem Spiel.

1. Programmablauf und Architektur

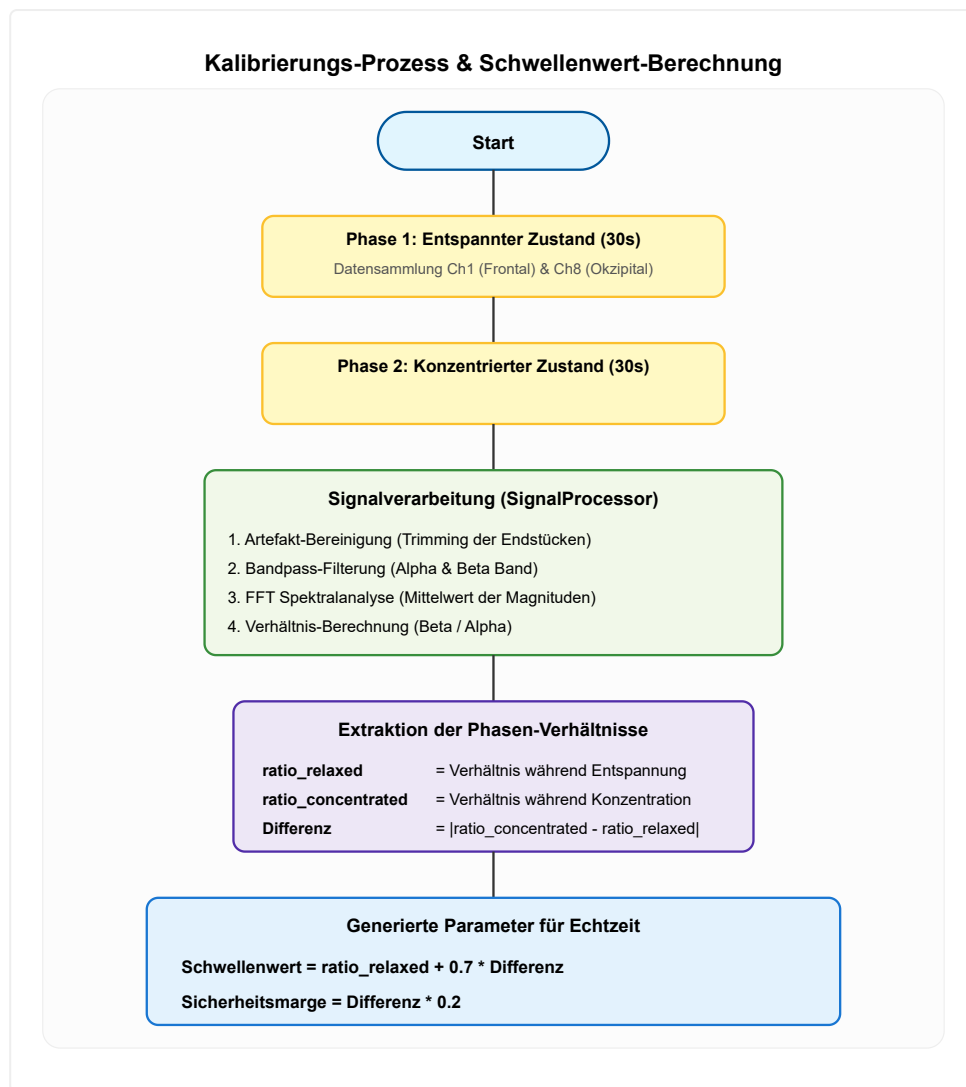
Im Folgenden wird der vollständige Ablauf des Programms, beginnend bei `Main.py`, beschrieben.

Programmablauf Übersicht:

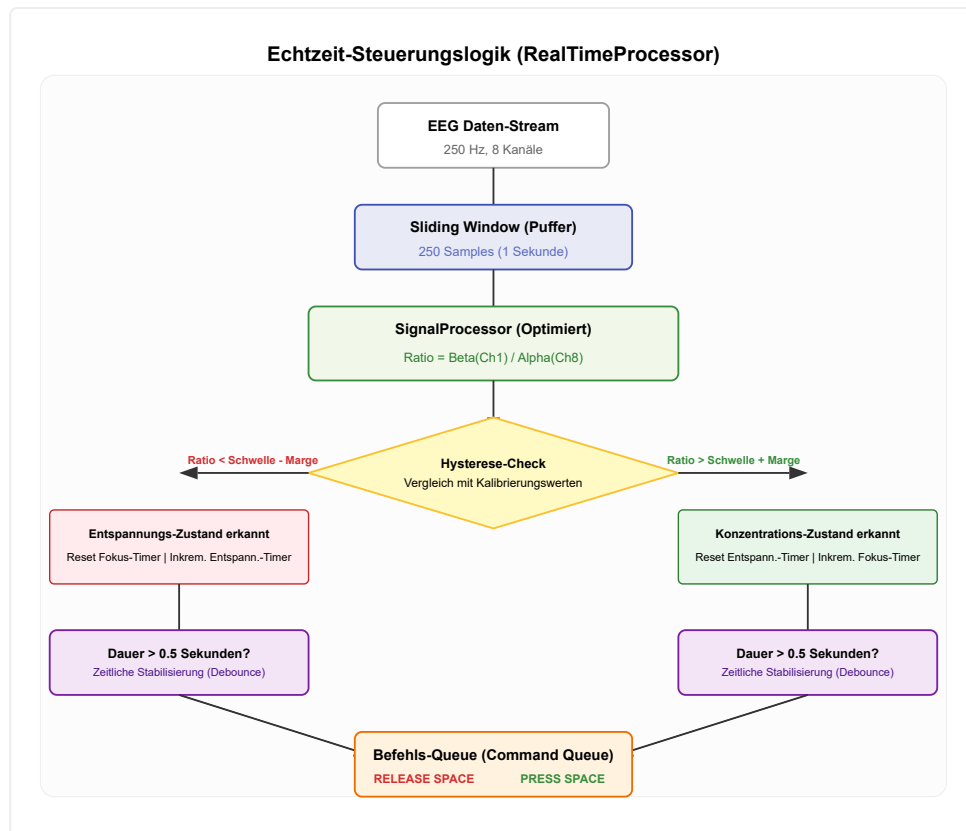
1. **Programmstart (`Main.py`):** Orchestriert den gesamten Ablauf.
2. **Kalibrierung:** Ermittelt individuelle Schwellenwerte für die Konzentrationserkennung.
3. **Echtzeit-Verarbeitung:** Analysiert kontinuierlich EEG-Daten und steuert die Spielfigur.

Erklärung des Programmablaufs:

1. **Programmstart (`Main.py`):**
2. Das Programm startet in der `Main` -Klasse
3. Die `main()` -Methode orchestriert den gesamten Ablauf
4. **Phase 1: Kalibrierung:**



5. CalibrationApp wird gestartet
6. Der Nutzer durchläuft zwei 30-sekündige Phasen (Relaxed und Concentrated)
7. Die Kalibrierung liefert 2 Werte zurück: threshold_ratio und margin_ratio
 - threshold_ratio : Ein gewichteter Schwellenwert basierend auf dem Beta/Alpha-Verhältnis beider Phasen.
 - margin_ratio : Eine Sicherheitsmarge (Hysterese), basierend auf der Differenz zwischen Relaxed und Concentrated.
8. **Phase 2: BCI-Prozess starten:**



9. RealTimeProcessor wird mit den Kalibrierungswerten initialisiert
10. Der Prozess startet als separater Hintergrundprozess (Multiprocessing)
11. Er analysiert kontinuierlich EEG-Daten und sendet Befehle über eine Queue
12. **Phase 3: Spiel starten:**
13. GameApp wird gestartet und erhält die command_queue
14. Die Spielschleife läuft mit 60 FPS
15. In jedem Frame:
 - Werden EEG-Befehle aus der Queue gelesen (falls vorhanden)
 - Tastendrücke simuliert (LEERTASTE drücken/loslassen)
 - Der aktuelle Spielzustand wird aktualisiert

2. Kalibrierungs-Logik im Detail

Der Kalibrierungsprozess ist entscheidend für die Bestimmung des individuellen Beta/Alpha-Verhältnis-Schwellenwerts.

Berechnungsschritte:

Phase 1 & 2: Datensammlung - Jede Phase dauert 30 Sekunden (ca. 7500 Samples bei 250 Hz) - Fokus auf Kanal 1 (Frontal) und Kanal 8 (Okzipital)

Schritt 1: Merkmals-Extraktion (Zentralisiert im SignalProcessor)

- **Filterung:** Butterworth-Bandpass-Filter 3. Ordnung (Kausal)
- **Power-Berechnung:** Spektralleistung (FFT-Magnitude) in spezifischen Bändern:
- **Alpha (8-12 Hz)** auf Kanal 8 (Okzipital)
- **Beta (13-30 Hz)** auf Kanal 1 (Frontal)
- **Normalisierung:** Die Leistung wird als **Mittelwert der Magnituden** über die Frequenz-Bins berechnet. **Wichtig:** Diese Normalisierung macht die Metrik unabhängig von der Fensterlänge. Dadurch sind Werte aus der 30-sekündigen Kalibrierungsphase direkt mit den 1-sekündigen Echtzeit-Fenstern vergleichbar.
- **Ratio-Berechnung:** $\text{Ratio} = \text{Beta_Ch1} / \text{Alpha_Ch8}$

Schritt 2: Schwellenwert-Berechnung - Für jede Phase wird ein repräsentatives Verhältnis berechnet (ratio_rel , ratio_con). - **Gewichteter Schwellenwert:** - $\text{threshold} = \text{ratio_rel} + \text{sensitivity} * (\text{ratio_con} - \text{ratio_rel})$ - Standard-Sensitivität: 0.7 (macht es etwas "schwerer", in den Bewegungs-Zustand zu kommen).

Schritt 3: Sicherheitsmarge (Margin) - Die Marge wird als Prozentsatz der Differenz berechnet: - $\text{margin} = |\text{ratio_con} - \text{ratio_rel}| * 0.2$ - Dient als Hysterese, um instabiles Schalten (Flickering) zu verhindern.

3. Echtzeit-Steuerungslogik (BCI)

Während das Spiel läuft, analysiert der `RealTimeProcessor` kontinuierlich EEG-Daten in einer performanten Loop:

Verarbeitungsschritte:

1. **Datenerfassung:** `EEGManager` ruft Samples ab.
2. **Sliding Window:** Ein 1-Sekunden-Puffer wird gepflegt.
3. **Optimierte Signalverarbeitung:**
 - Der `SignalProcessor` berechnet die Ratio.
 - **Optimierung:** Es werden nur die benötigten Kanäle (1 und 8) gefiltert und analysiert, nicht der gesamte EEG-Stream.
4. **Zeit-basierte Hystereselogik (Debounce):**
 - **Zustandserkennung:**

- `is_concentrated` : `ratio > threshold + margin`
 - `is_relaxed` : `ratio < threshold - margin`
 - **Bestätigungszeit:** Ein Zustandswechsel wird nur ausgelöst, wenn die Bedingung kontinuierlich für eine bestimmte Dauer (z.B. 0.5s) erfüllt ist.
5. **Befehlsausführung:** `PRESS SPACE` bei Konzentration, `RELEASE` bei Entspannung.
-

4. Überblick der Ordnerstruktur

- **Main.py** : Haupteinstiegspunkt, orchestriert den Übergang von Kalibrierung zu BCI-gesteuertem Spiel.
- **EEG/** : Kernlogik für Signalverarbeitung (Filterung, FFT) und Echtzeit-Multiprocessing.
- **Calibration/** : MVC-Implementierung der interaktiven Kalibrierungs-Anwendung.
- **RedLightGreenLight/** : Hauptanwendung des Spiels mit High-Level State Machine und grafischen Assets.
- **UIUtils/** : Wiederverwendbare Pygame-GUI-Komponenten (Buttons, Menüs).
- **TestData/** : Vor-aufgezeichnete EEG-Daten für die Entwicklung und Simulation ohne Hardware.
- **Tests/** : Automatisierte Skripte zur Validierung der Signalverarbeitungskette.